

VERANO INGENIERÍA

Procedimiento de Control de Versiones, Integración Continua y Flujo de Trabajo Colaborativo

Proyecto Verano Link

Integración Oracle Primavera Cloud (OPC) ↔ Hexagon Ecosys — Cliente: YPF

Campo	Detalle
Proyecto	Verano Link — Integración OPC ↔ Ecosys (Cliente YPF)
Versión del documento	1.0
Fecha de emisión	15 de julio de 2026
Elaborado por	Miguel Astaiza — Desarrollador
Aplica a	Equipo de desarrollo Verano Link (2 desarrolladores + administrador de DB)
Repositorio	GitHub — VeranoLink_YPF (público)
Estado	Vigente — de obligatorio cumplimiento para el equipo del proyecto

Tabla de contenido

Tabla de contenido	2
1. Objetivo y alcance	3
2. Contexto técnico del proyecto	3
3. Equipo y roles	3
4. Herramientas oficiales del proyecto	4
5. Estructura del repositorio.....	4
6. Repositorio público y manejo de información sensible	4
6.1 Credenciales — nunca en el código.....	4
6.2 Ofuscación de datos de ejemplo	5
7. Control de versiones de base de datos — Liquibase (capa Projects).....	5
8. Control de versiones de la aplicación APEX — APEXlang	5
9. Gestión de requerimientos y ajustes — GitHub Issues	6
10. Metodología de ramas (branching).....	6
10.1 Cuándo crear una rama nueva	6
10.2 Convención de nombres	7
10.3 Ciclo de vida de una rama	7
11. Flujo de Pull Request, revisión cruzada y aprobación	7
12. Integración continua — GitHub Actions	8
13. Despliegue a Stage y Producción (manual).....	8
13.1 Despliegue a Stage.....	8
13.2 Despliegue a Producción	8
14. Resumen del flujo completo, de punta a punta	9
15. Control de cambios de este documento	9

1. Objetivo y alcance

Este documento define el procedimiento estándar de control de versiones (base de datos, aplicación APEX y documentación), integración continua y flujo de colaboración en Git/GitHub que el equipo del proyecto **Verano Link** debe seguir para el desarrollo de la integración con el cliente **YPF**.

Aplica a los desarrolladores del proyecto y al responsable de infraestructura, quien aprueba las fusiones de código y administra los ambientes. Su cumplimiento es obligatorio a partir de la fecha de emisión de este documento.

2. Contexto técnico del proyecto

Resumen de la arquitectura y alcance funcional vigente, como referencia para entender por qué se definieron las prácticas de este procedimiento:

- Integración **batch nocturna vía REST** (sin tiempo real) entre Oracle Primavera Cloud (OPC) y Hexagon Ecosys, usando Verano Link como intermediario.
- Desarrollo **100% en PL/SQL** sobre Oracle Database, con Oracle **APEX 26.1.1** como interfaz de monitoreo para el cliente.
- Únicas dos acciones habilitadas hacia Ecosys: **CrearActividad** (PUT) y **ActualizarActividad** (POST), autenticadas con Basic Auth.
- Dos ambientes: **Stage** (**v1-ypf-stage.veranocloud.com.co**) y **Producción** (**v1-ypf-prod.veranocloud.com.co**), cada uno con su propia base de datos (servicios **YPF_VL_STGE** / **YPF_VL_PROD**) y su workspace APEX **VERANOLINK**.

3. Equipo y roles

Rol	Responsabilidades en este procedimiento
Desarrollador (x2)	Registran Issues, crean ramas, desarrollan en PL/SQL y APEX, exportan cambios (Liquibase / APEXlang), abren Pull Requests, ejecutan pruebas cruzadas sobre los cambios del compañero, ejecuta el despliegue manual a Stage y Producción.
Administrador DB	Administra los ambientes Stage/Prod, revisa y aprueba (o rechaza) los Pull Requests.

4. Herramientas oficiales del proyecto

Herramienta	Uso en el proyecto
GitHub (plan Free, repositorio público)	Control de versiones, Pull Requests, protección de rama principal, Issues.
SQLcl — Liquibase (capa "Projects")	Versionamiento de objetos de base de datos: tablas, vistas, paquetes, procedimientos.
Oracle APEX 26.1.1 — formato APEXlang	Versionamiento de la aplicación APEX, a nivel de componente/página (.apx).
GitHub Actions	Integración continua: validación automática de estructura en cada Pull Request.
GitHub Issues	Registro y trazabilidad de requerimientos y ajustes identificados.

5. Estructura del repositorio

El repositorio **VeranoLink_YPF** se organiza de la siguiente manera:

```

verano-link-ypf/
├── db/           -> proyecto SQLcl (Liquibase "Projects": build / stage / release)
├── apex/        -> export APEXlang de la aplicación (.apx, uno por componente)
├── docs/        -> este procedimiento y demás documentación del proyecto
├── .github/
│   └── workflows/ -> definiciones de los workflows de GitHub Actions
└── README.md

```

6. Repositorio público y manejo de información sensible

El repositorio del proyecto es **público**, por lo que no existe impedimento contractual para que el nombre de las empresas o la lógica de integración sean visibles públicamente.

Sin embargo, el carácter público del repositorio **no exime** al equipo de proteger la información técnica sensible. Las siguientes reglas son de cumplimiento obligatorio, independientemente de si el repositorio fuera público o privado:

6.1 Credenciales — nunca en el código

- Credenciales de los servicios REST (Ecosys, OPC) se gestionan mediante **APEX Web Credentials**: se configuran una única vez en el workspace y se referencian por nombre lógico desde **APEX_WEB_SERVICE**. El usuario y la contraseña reales no se escriben en ningún script.
- Credenciales de conexión a base de datos utilizadas por Liquibase se gestionan mediante **sustitución de propiedades** (variables de entorno o secretos de GitHub Actions), nunca como texto literal dentro de un changelog.

6.2 Ofuscación de datos de ejemplo

Cualquier dato de prueba o “semilla” (seed) incluido en scripts versionados (por ejemplo, sentencias **INSERT INTO** de ejemplo) debe usar valores sintéticos, nunca datos reales de producción:

Correcto (ofuscado, para el repositorio)	Incorrecto (dato real, nunca en el repositorio)
INSERT INTO OPC_ECOSYS_LOG (CONTRACT_ID, ACTIVITY_NAME) VALUES ('99999999', 'Actividad de ejemplo - 001');	INSERT INTO OPC_ECOSYS_LOG (CONTRACT_ID, ACTIVITY_NAME) VALUES ('20260505', 'Limpieza de terreno - 002');

Nota: aunque el contrato permite exponer el nombre de las empresas, la regla de nunca exponer credenciales ni datos reales de producción aplica sin excepción.

7. Control de versiones de base de datos — Liquibase (capa Projects)

El versionamiento de los objetos de base de datos (tablas, vistas, paquetes, procedimientos) se realiza con Liquibase, embebido en SQLcl, usando la capa **Projects**, que organiza el trabajo en tres fases:

Fase	Qué ocurre
Build	El desarrollador trabaja libremente en la base de datos (Stage/desarrollo): crea o modifica tablas, paquetes, procedimientos, etc.
Stage	Se ejecuta el comando de “stage” del proyecto: SQLcl genera y organiza automáticamente los scripts de DDL de lo creado/modificado (o se cura manualmente qué incluir).
Release	Se empaqueta lo “staged” en una entrega versionada; por debajo, SQLcl genera los changelogs de Liquibase correspondientes, listos para aplicar.

Pasos que sigue el desarrollador ante cualquier cambio de base de datos:

1. Realizar el cambio en la base de datos de desarrollo/Stage (fase Build).
2. Ejecutar la etapa de “Stage” del proyecto SQLcl para capturar el cambio como changelog.
3. Revisar el changelog generado dentro de la carpeta **db/** del repositorio.
4. Incluir el changelog en la rama de trabajo correspondiente (ver sección 10) y hacer commit.
5. La aplicación del changelog contra Stage/Producción se realiza en la etapa de despliegue (sección 13), no antes.

8. Control de versiones de la aplicación APEX — APEXlang

Al confirmarse que el ambiente corre Oracle APEX **26.1.1**, el versionamiento de la aplicación se realiza con **APEXlang**, el formato de exportación nativo introducido en APEX 26.1 (mayo de 2026), diseñado específicamente para control de versiones.

- Se utiliza la opción de exportación **Standard Export**, recomendada por Oracle para control de versiones (incluye la metadata de desarrollador necesaria).

- El resultado es **un archivo .apx por página o componente compartido** (en lugar de un único script SQL monolítico), lo que permite comparar (diff) y revisar cambios de forma granular en cada Pull Request.
- La app utiliza **IDs estáticos** como identificadores estables, necesarios para que las comparaciones entre versiones sean limpias y consistentes.

Nota: la importación de APEXlang, en esta versión, solo soporta reimportar la aplicación completa (no es posible importar una sola página de forma individual). Esto no afecta la revisión por componente en el Pull Request, pero sí implica que el despliegue siempre reimporta la app entera.

Pasos que sigue el desarrollador ante cualquier cambio en la aplicación APEX:

6. Realizar el cambio en el App Builder del workspace VERANOLINK (ambiente de desarrollo/Stage).
7. Exportar la aplicación en formato APEXlang (Standard Export).
8. Guardar el resultado en la carpeta **apex/** del repositorio, reemplazando únicamente los archivos .apx afectados.
9. Incluir los archivos en la rama de trabajo correspondiente y hacer commit.

9. Gestión de requerimientos y ajustes — GitHub Issues

Todo requerimiento nuevo, ajuste identificado o corrección se registra **como un Issue en GitHub antes de comenzar a codificar**. Esto da trazabilidad entre el código y la razón por la que existe cada cambio.

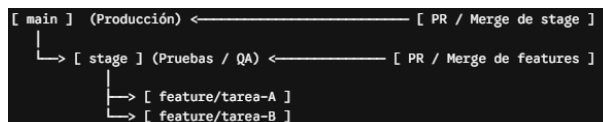
Información mínima de un Issue:

- Título breve y descriptivo.
- Tipo: requerimiento nuevo / ajuste / corrección.
- Descripción del comportamiento esperado.
- Ambiente afectado (Stage / Producción / ambos).

10. Metodología de ramas (branching)

Principio general: **una rama por unidad de trabajo** — es decir, un Issue equivale a una rama. No se acumulan cambios no relacionados entre sí en una misma rama.

Ramas Fijas: Desde el inicio de git es necesario crear una rama secundaria llamada “Stage”, de esta manera siempre existirá la rama principal main y la rama secundaria Stage.



10.1 Cuándo crear una rama nueva

Siempre que se inicie trabajo sobre un Issue: un requerimiento nuevo, un ajuste identificado o una corrección. Si el alcance de un cambio crece y empieza a tocar algo no relacionado con el Issue original, ese trabajo adicional se separa en un nuevo Issue y una nueva rama.

10.2 Convención de nombres

Prefijo	Uso	Ejemplo
feature/	Requerimiento nuevo	feature/12-crear-actividad-post
fix/	Corrección de algo que no funciona	fix/15-error-mapeo-fechas
chore/	Ajuste menor, refactor, no funcional	chore/18-ajuste-nombre-columna

Patrón general: **tipo/numero-issue-descripcion-corta**.

10.3 Ciclo de vida de una rama

10. Crear la rama a partir de **Stage** actualizado.
11. Desarrollar y hacer commits del cambio (base de datos y/o APEX, según corresponda).
12. Publicar (push) la rama al repositorio remoto.
13. Abrir un Pull Request hacia **Stage**, referenciando el número del Issue.
14. Tras la fusión (merge), eliminar la rama inmediatamente para mantener el repositorio limpio.

11. Flujo de Pull Request, revisión cruzada y aprobación

Ningún cambio llega a **Stage** y a **main** sin pasar por este flujo:

15. El desarrollador finaliza su rama y abre un Pull Request hacia **Stage**.
16. Se dispara automáticamente el workflow de GitHub Actions (sección 12), que valida la estructura de lo que se va a fusionar.
17. El otro desarrollador realiza la **prueba cruzada**: ejecuta el cambio contra el ambiente de Stage y valida el comportamiento funcional (¿la actividad se crea/actualiza correctamente en Ecosys?, ¿los datos quedan bien registrados?).
18. El desarrollador que probó documenta el resultado directamente como comentario en el Pull Request: qué se probó, contra qué ambiente, y el resultado obtenido.
19. El Administrador DB revisa el Pull Request (código, resultado de la prueba cruzada y resultado de GitHub Actions) y lo aprueba o solicita cambios.
20. La fusión a **Stage** solo se habilita cuando se cumplen ambas condiciones a la vez: aprobación del responsable de infraestructura y validación exitosa de GitHub Actions.
21. Una vez los cambios están validados y aplicados en **Stage** se abre un Pull Request hacia **main** para alinear los cambios.

Nota: al ser el repositorio público, GitHub Free permite configurar una regla de protección sobre la rama main que exige de forma técnica (no solo por acuerdo de equipo) la aprobación del Pull Request y el resultado exitoso de GitHub Actions antes de permitir la fusión.

12. Integración continua — GitHub Actions

GitHub Actions es el mecanismo de automatización que ejecuta validaciones en la nube de GitHub, disparadas automáticamente por eventos (como la apertura de un Pull Request), sin depender de que un desarrollador las ejecute manualmente en su equipo.

Su función en este proyecto es validar **estructura**, no lógica de negocio — esta última sigue siendo responsabilidad de la prueba cruzada (sección 11). Concretamente, el workflow definido en `.github/workflows/` se dispara en cada apertura o actualización de un Pull Request hacia `Stage` o `main`, y ejecuta:

- Validación de sintaxis y estructura de los archivos APEXlang (`.apx`) exportados, mediante `apex-validate` (SQLcl).
- Validación de estructura de los changelogs de Liquibase generados por la capa Projects (detecta problemas antes de que se intente aplicar cualquier cambio a un ambiente real).

Si cualquiera de estas validaciones falla, el Pull Request queda marcado como no apto para fusión, y el desarrollador debe corregir antes de continuar.

13. Despliegue a Stage y Producción (manual)

El despliegue continuo (CD) automatizado no es una prioridad actual del proyecto; el despliegue se realiza de forma **manual** por el responsable de infraestructura (o el desarrollador que este designe), inmediatamente después de cada fusión a `Stage` o `main`.

13.1 Despliegue a Stage

22. Aplicar los changelogs de Liquibase pendientes (`1b update`) contra la base de datos de Stage (`YPF_VL_STGE`).
23. Importar el export APEXlang más reciente sobre la aplicación del workspace VERANOLINK en Stage.
24. Validar en Stage que el cambio funciona correctamente antes de promoverlo a Producción.

13.2 Despliegue a Producción

25. Solo después de validar exitosamente en Stage.
26. Aplicar los mismos changelogs de Liquibase (`1b update`) contra `YPF_VL_PROD`.
27. Importar el mismo export APEXlang sobre la aplicación del workspace VERANOLINK en Producción.

14. Resumen del flujo completo, de punta a punta

Paso	Acción	Responsable
1	Registrar el requerimiento/ajuste como Issue en GitHub	Cualquier desarrollador
2	Crear rama a partir de main (feature/ fix/ chore/)	Desarrollador asignado
3	Desarrollar el cambio (PL/SQL y/o APEX)	Desarrollador asignado
4	Generar changelog (Liquibase Projects) y/o export APEXlang	Desarrollador asignado
5	Push de la rama y apertura de Pull Request	Desarrollador asignado
6	Validación automática (GitHub Actions)	Automático
7	Prueba cruzada sobre Stage	El otro desarrollador
8	Revisión y aprobación del Pull Request	Responsable de infraestructura
9	Fusión a Stage/main y eliminación de la rama	Responsable de infraestructura
10	Despliegue manual a Stage, validación, y luego a Producción	Desarrollador asignado

15. Control de cambios de este documento

Versión	Fecha	Autor	Descripción del cambio
1.0	15/07/2026	Miguel Astaiza	Versión inicial del procedimiento.
1.1	16/07/2026	Sebastian Castro	Manejo de rama Stage y main